```c
#include "gen_table.h"
#include "linked_list.h"

gentabl_t* create_empty_gentabl(int n) {
 gentabl_t* gt = malloc(sizeof(gentabl_t));
 gt->num_elts = 0;
 gt->size = n;

 gt->decomps = malloc(gt->size*sizeof(list_t*));
 for (int i = 0; i < gt->size; i++) {
  gt->decomps[i] = create_empty_list();
 }
 return gt;
}

bool is_empty_gentabl(gentabl_t* gt) {
 assert(gt != NULL);
 return (gt->size == 0);
}

void clear_gentabl(gentabl_t* gt) {
 for (int i = 0; i < gt->size; i++) {
  free_list(gt->decomps[i]);
 }
 free(gt->decomps);
 free(gt);
}

bool is_empty_index(gentabl_t* gt, int index) {
 assert(index < gt->size);
 list_t* to_check = gt->decomps[index];
 assert(to_check != NULL);
 return (is_empty(to_check));
}

list_t* get_copy_index_val(gentabl_t* gt, int i) {
 assert(!is_empty_index(gt, i));
 list_t* copy = deep_copy(gt->decomps[i]);
 return copy;
}

void copy_at_index(gentabl_t* gt, int index, list_t* val) {
 assert(gt != NULL);
 assert(index < gt->size);
 if (gt->decomps[index] != NULL) {
  free_list(gt->decomps[index]);
 }

 gt->decomps[index] = deep_copy(val);
}

void del_index(gentabl_t* gt, int index) {
 assert(gt != NULL);
 assert(index < gt->size);
 list_t* to_free = gt->decomps[index];
 free_list(to_free);
 gt->decomps[index] = create_empty_list();
}

void show_all_gentabl(gentabl_t* gt) {
 for (int i = 0; i < gt->size; i++) {
  printf("[%d]: ", i);
  show_values(gt->decomps[i]);
  printf("\n");
 }
}
```

```
void free_gentabl(gentabl_t* gt) {
 assert(gt != NULL);
 for (int i = 0; i < gt->size; i++) {
  free_list(gt->decomps[i]);
 }
 free(gt->decomps);
 free(gt);
}
```